DAY 5 · WORKED EXAMPLE

Guided LLM Prompt Chain in Google Colab

Recreate a full machine-learning notebook using a controlled LLM prompt sequence and a cloud notebook environment.

Input file

classification_workshop.csv
220 process runs
Target: defect

LLM role

Draft notebook cells
Explain code
Add human checks

Colab role

Run code in browser
Upload dataset
Share notebook

Human role

Audit assumptions
Validate metrics
Approve claims

Evidence → Prompt → Google Colab notebook → Results → Model card → Human validation

Worked example overview

What participants produce by the end of the Colab-guided prompt-chain demonstration

Colab-ready

Input file

classification_workshop.csv

Inputs:
- temperature_c
- pressure_kpa
- speed_mm_s
- vibration_mm_s

Target: defect

LLM task

Generate notebook-ready markdown and code.

Use exact column names.

Add a human verification check after each notebook section.

Colab task

Create notebook.
Upload CSV.
Run cells.
Restart runtime and run all.
Share or download final notebook.

Success criterion

The notebook should not just run. Faculty must be able to defend every modeling choice, metric, and conclusion.

Final product: runnable Colab notebook + prompt-response log + baseline/model comparison + model card

Google Colab setup

Setup

Start with a clean browser-based notebook so every participant can run the same workflow

Step 1 — Open Colab

Go to colab.research.google.com and create a new notebook. Rename it: Day5_LLM_Generated_ML_Pipeline.ipynb

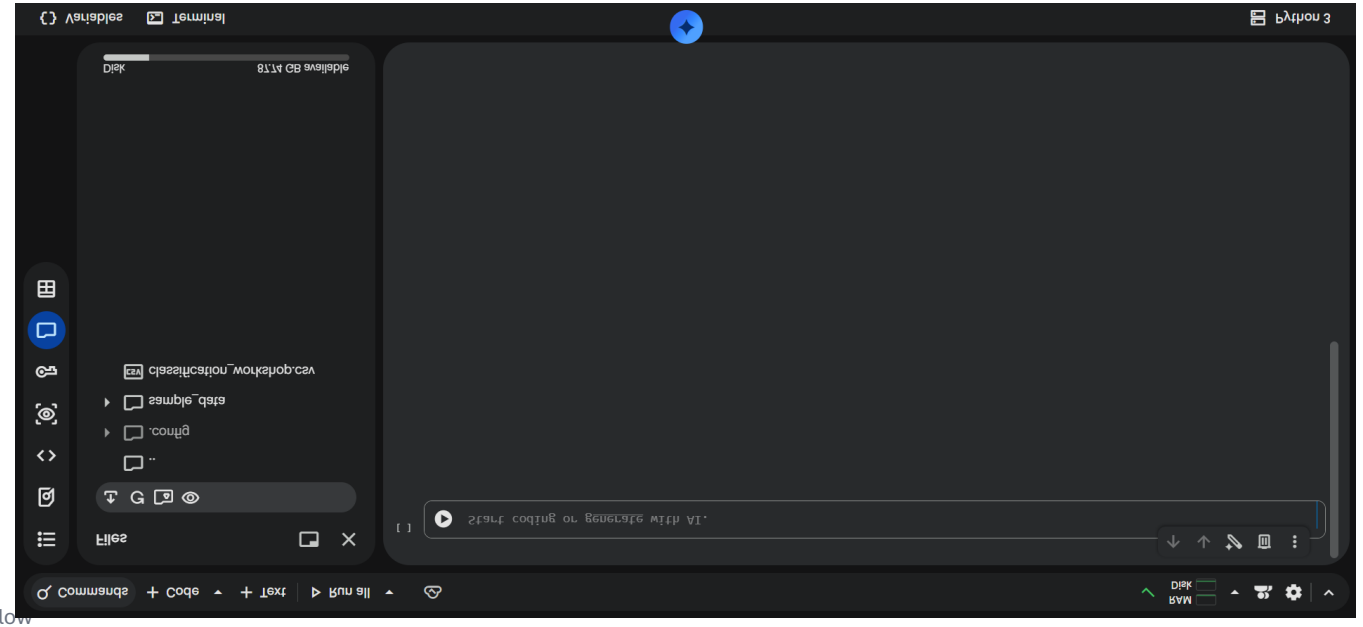
Step 2 — Upload dataset

Use the file pane or run a file-upload cell. Upload classification_workshop.csv into the Colab runtime.

Step 3 — Confirm runtime

Runtime → Restart session, then run cells from the top. Keep random_state=42 for consistent class results.

https://colab.research.google.com/



Prompt-chain map in Colab

Seven short prompts replace one vague request

7 prompts

0	1	2	3	4	5	6	7
Guardrails	Frame	Audit	Leakage	Features	Pipeline	Evaluate	Model card

For each prompt

Collect the exact prompt, the LLM-generated notebook section, and a human audit note: accept, revise, or reject.

Colab notebook habit

Paste generated code into one small cell at a time. Run the cell before continuing to the next prompt.

Human checkpoint

Do not continue if the LLM invents columns, ignores the baseline, fits preprocessing too early, or overclaims causality.

Prompt 0 — Establish role and guardrails

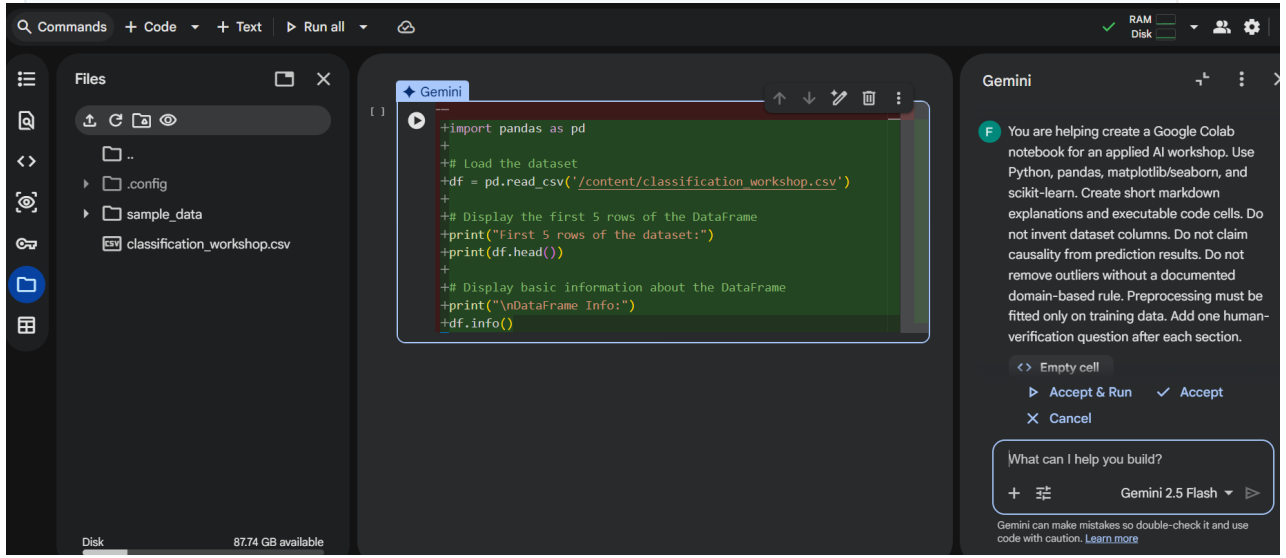
Use this before generating any notebook code

Copy-paste

You are helping create a Google Colab notebook for an applied AI workshop.

Use Python, pandas, matplotlib/seaborn, and scikit-learn.
Create short markdown explanations and executable code cells.

Do not invent dataset columns.
Do not claim causality from prediction results.
Do not remove outliers without a documented domain-based rule.
Preprocessing must be fitted only on training data.
Add one human-verification question after each section.



Worked response

The LLM should acknowledge constraints and ask for dataset name and column list before writing code.

Reject response

Do not accept a response that immediately invents variables, deploys the model, or writes code using unknown columns.

Colab checkpoint

Open a new text cell titled “Prompt log.” Paste Prompt 0 and the LLM response. This creates an auditable record of how the notebook was generated.

Prompt 1 — Problem framing

Frame

Generate the first Colab markdown and setup code cells

Dataset: classification_workshop.csv
Columns: temperature_c, pressure_kpa, speed_mm_s, vibration_mm_s, defect

Create notebook-ready markdown and code that identify: unit of observation, inputs, target, model task, baseline, and metric.

Element	Worked example
Unit	One process run
Inputs	temperature, pressure, speed, vibrat...
Target	defect: 0 = acceptable; 1 = defective
Task	Binary classification
Baseline	Predict majority class
Metric	Recall/F1, not accuracy alone

Colab output

Create one markdown cell for the framing table and one code cell defining features, target, and positive class.

Human check

Are all inputs available before final inspection? Is “defect” defined consistently?

Prompt 2 — Data audit and visualization

Use Colab to verify the dataset before modeling

Audit

Generate notebook cells to load the CSV and report:
- df.head(), df.shape, df.dtypes
- missing values and duplicates
- target class balance
- summary statistics
- count plot of defect
- scatter or box plots for variables

Worked audit

Rows: 220
Accept: 176
Defect: 44
Missing: 0

Evidence

Imbalance is present. A majority baseline can reach 80% accuracy while finding no defects.

```
df.shape
df.dtypes
df.isna().sum()
df["defect"].value_counts(normalize=True)
```

Human check

Did the LLM use actual columns? Do the plots show class balance and feature ranges rather than decorative figures?

Prompt 3 — Leakage and availability review

Trust check

Provide the prediction time so the LLM can reason about leakage

Prediction time: before final quality inspection.
Inputs: temperature_c, pressure_kpa,
speed_mm_s, vibration_mm_s.
Target: defect.

Create a table: variable, role, available at prediction time?, leakage risk?, decision.

Variable	Risk	Decision
temperature_c	Low	Keep
pressure_kpa	Low	Keep
speed_mm_s	Low	Keep
vibration_mm_s	Low	Keep
defect	Target	Do not use as input

Colab note

Place the leakage table in a markdown cell before the modeling code. It documents what the model is allowed to know.

Human check

The LLM cannot know timing unless you provide it. Add workflow context before accepting keep/remove decisions.

Prompt 4 — Feature engineering with discipline checks

Features

Use AI to suggest features, not to approve them automatically

Suggest candidate features using temperature, pressure, speed, and vibration.

Separate into:
1. domain-informed
2. mathematically convenient
3. risky or possibly leaky

Provide code only for justified features.
Do not claim causality.

Feature	Status	Rationale
temperature_devia...	Clarify	Need nominal temperatu...
pressure_deviation	Clarify	Need target pressure
vibration_speed_r...	Test	May reflect instabilit...
future inspection...	Reject	Outcome-derived leakage

Worked decision

For the simplified Colab demo, keep raw features only. Save engineered features as next-step hypotheses.

Teaching point

AI-generated features are hypotheses. They are not evidence until tested and validated.

Prompt 5 — Generate the reproducible Colab pipeline

Code cells

The key technical cell: split, baseline, pipeline, predict, evaluate

```
X = df[features]
y = df["defect"]

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.25, stratify=y, random_state=42
)

baseline = np.full(len(y_test), y_train.mode()[0])

model = Pipeline([
   ("scale", StandardScaler()),
   ("classifier", LogisticRegression(max_iter=1000))
])
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

Audit checks

- ✓ Split before fitting
- ✓ Baseline same test set
- ✓ Scaler inside pipeline
- ✓ Fixed random_state

Revise if

- ✗ scaler fit before split
- ✗ training-set evaluation
- ✗ no baseline
- ✗ invented features

Colab practice

Run the code cell, then immediately add a markdown cell titled “Human audit.” Record whether the pipeline passed the checks.

Categorical reminder

For categorical features, use ColumnTransformer and OneHotEncoder. This simplified file has numeric inputs only.

Worked model results

Results

The output the Colab notebook should reproduce

Baseline accuracy

0.800
predicts every run acceptable

Baseline recall

0.000
finds no defects

Model accuracy

0.855
slightly higher overall

Model precision

1.000
no false alarms

Model recall

0.273
misses most defects

	Predicted ...	Predicted ...
Actual acceptable	44	0
Actual defect	8	3

Model interpretation

The model is conservative: when it flags a defect it is correct, but it detects only 3 of 11 actual defects. It should not replace inspection.

Prompt 6 — Evaluation and error analysis

Metrics

Separate direct evidence from unsupported claims

Interpret these results for a process engineer.
Separate:
- direct evidence
- reasonable interpretation
- unsupported conclusions
- most costly error
- recommended next analysis

Missed defects are more costly than false alarms.

Category	Worked response
Direct eviden…	Accuracy rose from 0.800 to 0.855; re…
Interpretation	Model is cautious and misses many def…
Unsupported	Process variables cause defects
Costly error	False negatives: 8 missed defects
Next analysis	Threshold sweep, class weighting, cro…

Human check

If the LLM says the model is “ready for deployment,” revise. Evidence supports limited screening value only.

Prompt 7 — Completed model card

Model card

Final documentation section for the Colab notebook

Create an initial model card with:
- intended use
- out-of-scope use
- dataset and target
- baseline result
- model result
- primary error pattern
- supported conclusion
- unsupported conclusion
- limitations
- recommended next analysis

Field	Worked entry
Intended use	Teaching example; inspection prioritiza...
Out-of-scope	Automatic part approval or production d...
Result	Acc 0.855, precision 1.000, recall 0.273
Error pattern	Missed 8 of 11 actual defects
Do not claim	Variables cause defects
Next step	Tune threshold and collect more defect ...

Colab deliverable

One evidence-grounded paragraph plus the structured model-card table in markdown.

Closing synthesis

AI drafts the artifact; faculty validate the workflow

Takeaways

Data evidence

Colab runs the notebook and exposes the actual tables, plots, and metrics.

LLM drafts

The LLM creates a first-pass notebook scaffold and explanation.

Metrics evaluate

Baseline, confusion matrix, and recall reveal whether the model is useful.

Human validates

Faculty approve assumptions, code, metrics, and claims.

Final message: Google Colab makes the workflow easy to run, but trustworthy AI use still depends on human-validated evidence and clear boundaries.

Do not ask: “Can AI build the notebook?” Ask: “Can we inspect, run, and defend every cell the AI helped create?”